FIT COMPETITION 2026

TRACK III: Identity and Data Exploitation

Leakage-Aware Phishing URL Detection with Deduplication-First Gradient Boosting

Written by:

Riantama Putra

Athilla Zaidan Zidna Fann

Nicholas Wise Saragih Sumbayak

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

2026

Abstract

The rapid digitization of humanitarian operations has expanded the cyber attack surface threatening vulnerable populations, with phishing remaining the most pervasive entry vector against digital identity, financial resources, and system integrity. This report presents a robust, leak-free tabular classification pipeline for detecting phishing URLs using strictly intra-URL lexical and structural features. The pipeline analyzes 247,950 URLs described by 41 integer and binary features. A central contribution is the rigorous identification and mitigation of tabular data leakage. Strict pre-split deduplication and 5-fold stratified cross-validation establish an intellectually honest evaluation paradigm. The analysis further contrasts Logistic Regression baselines against state-of-the-art gradient boosting frameworks (LightGBM, XGBoost, CatBoost), addressing security-aware threshold tuning, precision-recall trade-offs in imbalanced datasets, and feature attribution via SHapley Additive exPlanations (SHAP). Ultimately, this exhaustive report synthesizes cutting-edge machine learning optimization methodologies with the pressing, real-world imperative of global humanitarian cybersecurity, offering a reproducible blueprint for deploying trustworthy, highly interpretable, and computationally lightweight threat detection systems in high-stakes, resource-constrained environment

Table of Contents

Abstract **1**
Table of Contents **2**
Chapter I Introduction **4**
1.1. Background 4
1.2. Problem Statement 4
1.3. Objectives 5
1.4. Benefits 6
Chapter II Literature Review **7**
2.1. Data Science Concepts 7
Tabular Data and Lexical Features 7
Data Leakage 7
Evaluation Metrics and Interpretability 7
2.2. Machine Learning Methods 8
Logistic Regression 8
Gradient Boosting Frameworks: 8
2.3. Related Topics 8
Cybersecurity in the Humanitarian Sector 8
Advancements in Machine Learning for Phishing Detection 9
Tabular Data Leakage and Methodological Integrity 9
Chapter III Methodology **10**
3.1. Dataset Overview 10
3.2. Data Description 10
3.3. Data Preprocessing 11
3.4. Model and Method 14
3.5. Model Evaluation 14
Chapter IV Results and Discussion **16**
4.1. Results 16
4.2. Data Insights 20
4.3. Model Interpretation 21
Chapter V Conclusion and Recommendations **22**
5.1. Conclusion 22
5.2. Recommendations 23
References **24**
Appendix **25**
A. Source Code 25
B. Additional Figures 25
C. Full Feature List by Category 25

Chapter I Introduction

1.1. Background

Humanitarian organizations have rapidly digitized their operations, utilizing cloud databases, biometric tracking, and geospatial tools to improve the speed and efficiency of crisis response. However, this transformation has inadvertently made them highly lucrative targets for cybercriminals and state-sponsored actors. These organizations operate within a paradox: they are "target-rich" due to the vast amounts of sensitive donor and beneficiary data they hold, yet chronically "cyber-poor" due to financial constraints that force them to prioritize physical aid over digital defense.

This "cyber inequity" leaves the humanitarian sector critically vulnerable, particularly during global emergencies. The World Economic Forum’s 2026 report highlights that 51% of NGOs lack the necessary resources to combat digital threats. Real-world consequences are devastating, ranging from ransomware attacks like the one on Kido International that exposed the sensitive data of 8,000 children, to the 135.5 billion malicious network requests blocked by Cloudflare during the 2020 racial justice protests.

When a vulnerable population's digital footprint is compromised, the consequences escalate beyond financial theft to potential physical harm or targeted persecution. Initiatives like the FIT Competition 2026 challenge researchers to build resilient, data-driven systems to protect these populations. Reliable, automated phishing detection serves as the critical first line of defense to secure NGO networks and ensure the integrity of humanitarian databases.

1.2. Problem Statement

While sophisticated deep learning models that analyze the visual or full code structure of a webpage are highly effective at detecting phishing, they are computationally prohibitive for the resource-constrained field offices of most NGOs. Extracting and analyzing intra-URL lexical features

directly from the raw text string offers a highly scalable, low-latency alternative that can run on standard hardware without initiating remote server connections.

This research evaluates a dataset of 247,950 URLs, mapping each to a single binary target (0 for legitimate, 1 for malicious) and 41 numerical features like string length and special character counts. However, mapping highly variable text strings into simple integer counts introduces a severe methodological flaw: tabular data leakage. Because many distinct URLs map to identical integer counts, thousands of mathematically indistinguishable rows are created within the dataset.

If a naive random split is applied to separate the data, these duplicate feature vectors contaminate both the training and testing sets. Instead of learning generalized decision boundaries to detect new threats, the machine learning algorithm simply memorizes the labels of the duplicates, artificially inflating performance metrics. The core problem is engineering a rigorously secure pipeline that explicitly eliminates this structural duplication to achieve an honest evaluation of the model.

1.3. Objectives

The primary objective of this research is to architect a secure, leak-free machine learning pipeline capable of differentiating malicious phishing URLs from legitimate ones. To achieve this, the methodology enforces strict data leakage prevention by programmatically identifying and permanently eliminating duplicate feature vectors across the entire 247,950-record dataset before any train-test splitting occurs.

A secondary goal is establishing a robust, reproducible experimental design. The pipeline uses a strict global random seed, isolates a 20% held-out test set, and applies a 5-fold Stratified Cross-Validation strategy to perfectly maintain minority class proportions. It also benchmarks a linear Logistic Regression model against state-of-the-art gradient boosting frameworks (LightGBM, XGBoost, CatBoost) using

standardized hyperparameter configurations and avoiding validation-based early stopping.

Finally, the research prioritizes security-aware evaluation and interpretability. Because relying solely on accuracy is insufficient for high-stakes humanitarian work, the pipeline evaluates models using ROC-AUC, Average Precision, and F1-Scores to carefully calibrate decision thresholds. Furthermore, it deploys SHAP to decode the algorithms, transparently revealing exactly which lexical anomalies drive the identification of cyber threats.

1.4. Benefits

From an operational perspective, this pipeline provides under-resourced NGOs with a highly accurate, computationally lightweight tool to continuously screen incoming communications and web traffic. Because it relies strictly on lexical features, the resulting models demand minimal processing overhead and can be seamlessly integrated into edge devices or local network firewalls commonly used in remote field operations.

By neutralizing phishing vectors at the network perimeter, humanitarian organizations can drastically reduce the risk of ransomware infections and protect both donor financial databases and the digital identities of vulnerable refugees. This proactive defense ensures that vital, life-saving logistical operations are not paralyzed by digital sabotage during critical crisis response windows.

Methodologically, this research corrects pervasive evaluation flaws found in contemporary cybersecurity literature. By exposing the mathematical mechanics of tabular data leakage and enforcing strict deduplication, the pipeline sets an uncompromising standard for machine learning validation. It proves to the data science community that algorithmic transparency, computational efficiency, and rigorous experimental integrity can be achieved simultaneously.

Chapter II Literature Review

2.1. Data Science Concepts

Tabular Data and Lexical Features

In cybersecurity, relying on complex payload analysis (e.g., rendering full HTML or CSS) is often hampered by latency constraints and attacker cloaking techniques. As a lightweight alternative, modern pipelines extract "lexical features", such as integer counts of specific characters (url_length, number_of_dots_in_url), binary flags (having_repeated_digits_in_url), and continuous string statistics, directly from the URL. This translates raw text into a standard tabular matrix ideal for machine learning.

Data Leakage

A critical vulnerability in machine learning evaluation is data leakage, which occurs when a model is exposed to test-set information during the training phase, artificially inflating its performance. In tabular URL analysis, this typically manifests as vector duplication. Because non-injective mapping collapses distinct URLs into identical feature vectors, naive random splitting guarantees that identical rows will contaminate both the training and test sets. Without strict global deduplication prior to evaluation, a model simply memorizes these vectors rather than learning generalizable decision boundaries.

Evaluation Metrics and Interpretability

For highly imbalanced cybersecurity data, relying on basic accuracy is deceptive. Robust evaluation requires metrics like Precision, Recall, F1-Score, and the Precision-Recall Area Under the Curve (PR-AUC) to balance the trade-off between missing threats and raising false alarms. Furthermore, to ensure operational transparency, frameworks like SHAP (SHapley Additive exPlanations) are utilized to explicitly decode which lexical anomalies are driving the algorithmic predictions.

2.2. Machine Learning Methods

The pipeline systematically benchmarks several algorithms suited for tabular classification.

Logistic Regression

Acting as a foundational, highly interpretable baseline, Logistic Regression utilizes a linear equation to model probabilities. Because it is highly sensitive to input scale (e.g., mixing large URL lengths with tiny binary flags), it requires rigorous feature standardization using tools like StandardScaler.

Gradient Boosting Frameworks:

To capture complex, non-linear relationships hidden within structural anomalies, state-of-the-art gradient boosted decision trees (GBDTs) are utilized. These frameworks build sequential trees to correct the residual errors of preceding ones.

1. LightGBM: Known for its rapid, leaf-wise tree growth, making it highly efficient for large datasets.
2. XGBoost: Utilizes level-wise growth and applies strict regularization (L1/L2 penalties) to prevent the model from overfitting to statistical noise.
3. CatBoost: Specifically engineered to handle tabular datasets by utilizing symmetric (oblivious) trees, maximizing inference speed and baseline accuracy.

2.3. Related Topics

Cybersecurity in the Humanitarian Sector

Humanitarian NGOs operate in volatile geopolitical spaces and manage vast databases of vulnerable populations, yet they frequently lack dedicated digital security funding. This dynamic makes them "target-rich, but cyber-poor". Recent literature emphasizes that deploying lightweight, scalable threat detection systems, such as those analyzing raw URL data at the network edge, is essential for proactively defending against phishing and ransomware without diverting capital from physical aid efforts.

Advancements in Machine Learning for Phishing Detection

The shift from static blocklists to dynamic machine learning analysis has revolutionized phishing detection. Extensive academic evaluations consistently demonstrate that ensemble methods, specifically XGBoost and Random Forest, yield the highest detection accuracy with minimal false positives when analyzing lexical features. By classifying URLs strictly based on character counts and structural irregularities, these models remain resilient against zero-day phishing campaigns.

Tabular Data Leakage and Methodological Integrity

Recent scholarship has issued urgent warnings regarding methodological flaws in cybersecurity machine learning. Studies highlight that data leakage from improper preprocessing or dataset duplication can falsely push algorithmic recall to near 100%, yielding deceptive models that fail catastrophically in production. By rigorously isolating duplication vectors and enforcing leak-free, mathematically sound cross-validation procedures, this pipeline directly addresses the academic reproducibility crisis, proving that robust evaluation methodology is far more critical than raw architectural complexity.

Chapter III Methodology

3.1. Dataset Overview

The dataset is a published, committee provided corpus of Uniform Resource Locators, each labeled as either legitimate or phishing and described exclusively by intra-URL features, that is, features computed from the URL string itself with no page content, document object model, HTML, or styling information [1]. The domain is digital security, and the task is to distinguish malicious links from benign ones early enough to block access before identity theft can occur. All facts reported below were derived directly from the provided data file rather than from any external description. We note only that published descriptions of the corpus disagree on which class count corresponds to phishing, which is one further reason to treat the data file itself as the ground truth.

3.2. Data Description

The corpus contains 247,950 records described by 42 columns: 41 feature columns and one target column. There are no missing values anywhere in the table and there is no identifier column. Of the 41 features, 39 are integer valued and three are continuous floating point values, namely the average subdomain length and the Shannon entropy of the URL and of the domain. Fifteen of the features are binary zero or one flags.

The features fall into natural groups. URL level lexical counts, fifteen in total, describe the link as a whole, including its length and the number of dots, digits, special characters, hyphens, underscores, slashes, question marks, equals signs, at signs, dollar signs, exclamation marks, hashes, and percent signs, together with a flag for repeated digits. Domain level features, eight in total, describe the registered domain, including its length, the number of dots and hyphens, and the presence and number of special characters and digits. Subdomain level features, eleven in total,

describe the subdomain structure, including the number of subdomains, the average subdomain length, and several presence and count flags. Five further features describe the presence of a path, the path length, and the presence of a query, a fragment, and an anchor. Two features capture the character entropy of the URL and of the domain. A complete grouped list is provided in the appendix.

The target column, named Type, takes the value zero for legitimate URLs and one for phishing URLs, and phishing is treated as the positive class throughout. In the full corpus the classes are close to balanced: 128,541 URLs, or 51.84 percent, are legitimate, and 119,409 URLs, or 48.16 percent, are phishing, a majority to minority ratio of 1.076, so phishing is in fact the minority class. Figure 1 shows the class balance both in the full corpus and after the deduplication described below.

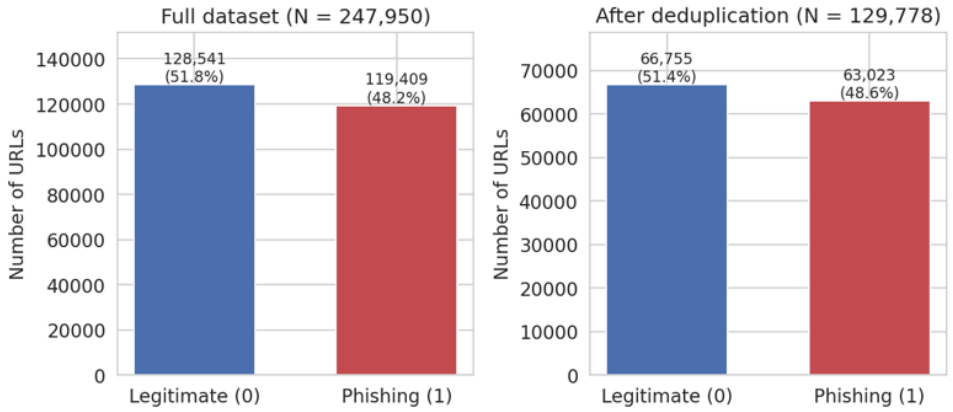


Figure 1. Class balance in the full corpus and after deduplication. The classes remain close to balanced in both cases.

3.3. Data Preprocessing

Preprocessing was deliberately minimal, and every decision was grounded in what the data showed rather than applied by default. Appropriate preprocessing includes knowing when not to apply a technique, and several of the decisions below are decisions not to transform the data.

Missing Values

There are none, so no imputation step is required. This was verified directly on the data.

Duplication and Deduplication

This is the central decision of the entire study. The data contains 118,172 exact duplicate rows, which is 47.66 percent of the corpus. This is a direct consequence of the coarse integer encoding: because each URL is reduced to a vector of counts and flags, many distinct URLs map onto the same vector. After exact duplicates are removed, 129,778 distinct rows remain, corresponding to 129,563 unique feature vectors when the label is ignored. Deduplication is performed before any train or test split, because otherwise identical vectors would appear in both partitions and inflate every metric. Chapter IV quantifies that inflation directly.

Label Conflicting Vectors

A small set of 215 feature vectors, spanning 848 rows or about 0.34 percent of the corpus, appear with both labels. These are short and structurally simple URLs, with a median URL length of 27 characters against 49 overall, whose coarse features cannot disambiguate the two classes. They are retained rather than removed, because they represent an irreducible noise floor that honestly caps the achievable accuracy, and because they are the only vectors that can legitimately appear on both sides of the split.

Constant and Near-constant Features

Seven features take a single value across every row and are therefore perfectly uninformative: the presence of a dot, a hyphen, special characters, and repeated digits in the subdomain, the average number of dots and of hyphens in the subdomain, and the presence of a path. About fourteen features are at least 99 percent a single value. These were retained, because a decision tree never splits on a constant feature and the standardizer used for the linear model maps a constant column to all zeros,

so the features are inert for every model considered and cannot affect any result. A variance threshold filter would remove exactly these seven features; we note the option but do not apply it, since it changes nothing.

Encoding and Normalization

No categorical encoding is required, because the feature matrix is already entirely numeric. Standardization is applied only to the logistic regression baseline and is fit on the training rows alone, so that no information from the evaluation data influences the scaling. The tree models are scale invariant and are trained on the raw features.

Feature Engineering

Four ratio features were constructed and tested honestly: digit density, special character density, hyphen density, and a subdomain depth ratio, each motivated by the intuition that the density of suspicious characters may carry more signal than their raw count. Adding these features to LightGBM under the same leak free cross validation changed the out-of-fold F1 from 0.9000 to 0.8998, a change of -0.0002, which is below the pre set adoption threshold of 0.002 and well within the fold to fold noise of the estimate. The features were therefore not adopted, and the raw 41 feature set was kept. The negative result is expected, because the trees already receive the raw counts and the URL length and can recover any such ratio internally through successive splits. We report this outcome in full rather than discarding it.

Class Balance

No resampling, class weighting, or synthetic oversampling was applied, because after deduplication the classes sit at about 49 to 51 and are already balanced. Applying a balancing technique here would inject synthetic or reweighted points and distort the calibration of predicted probabilities for no measurable benefit.

From the deduplicated data, a stratified 80 to 20 hold out split was drawn, producing 103,822 training rows and 25,956 test rows, both at a

phishing rate of 0.4856. Inside the training portion, stratified five fold cross validation was used, with validation folds of about 20,764 to 20,765 rows each. A check confirmed that only 77 feature vectors appear in both the training and test partitions, and that all of them are label conflicting vectors, so no identical pairing of features and label is shared across the split.

3.4. Model and Method

The choice of model family follows from the data rather than from any external benchmark. The 41 features are heterogeneous numeric counts and flags whose relationship to the label is strongly nonlinear and interaction driven: phishing risk depends not on any single count but on combinations, such as a long URL together with high entropy and a query string. Gradient-boosted decision trees capture such interactions directly, which is why LightGBM, XGBoost, and CatBoost are compared against a standardized logistic regression baseline whose role is to measure the value of that nonlinear capacity. A fixed, modest number of boosting rounds is used rather than early stopping, and this is a deliberate methodological guard. Early stopping would let each fold choose its stopping point by reading the very validation rows used to compute the out-of-fold score, which biases that score upward, whereas a fixed budget keeps the estimate clean and does not overfit on a dataset of this size. A soft voting average of the three boosting models is included, but only as a diagnostic to test whether averaging helps, not as a presumed winner.

3.5. Model Evaluation

Each model is evaluated with the full metric set: accuracy, and precision, recall, and F1 for the phishing class, together with ROC AUC and PR AUC. Every metric is reported both on the out-of-fold cross validation predictions, which are used for model comparison and selection, and on the held out test set, which is never seen during training. Confusion

matrices and ROC and precision-recall curves are produced for the final model. The agreement between the boosting models is examined by correlating their predicted probabilities, which informs whether an ensemble can help.

Using the metrics defined in Section 2.1, model selection is governed by the out-of-fold F1 on the phishing class, $\theta^* = \text{argmax}_\theta F1_{oof}(\theta)$. The operating threshold is chosen by the same principle, $\tau^* = \text{argmax}_\tau F1_{oof}(\tau)$, and is then applied unchanged to the held-out test set, so the operating point is selected without ever consulting the test labels.

Chapter IV Results and Discussion

4.1. Results

The final evaluation was conducted on the held-out test set using the deduplicated and leak-free split described in Chapter III. Phishing URLs are treated as the positive class, so precision, recall, and F1-score are interpreted from the perspective of phishing detection. This is important because in a security context, failing to detect a phishing URL is more harmful than incorrectly warning against a legitimate one.

The model comparison shows that LightGBM is the strongest candidate. As shown in Figure 2, LightGBM achieves the best balance across F1, ROC AUC, and PR AUC. On the held-out test set, LightGBM reaches an F1-score of 0.8991, ROC AUC of 0.9656, accuracy of 0.9045, precision of 0.9231, recall of 0.8763, and PR AUC of 0.9689. XGBoost, CatBoost, and the soft-voting ensemble also perform strongly, but none of them surpass LightGBM. The Logistic Regression baseline performs substantially worse, with an F1-score of 0.7779, showing that the classification boundary is not purely linear.

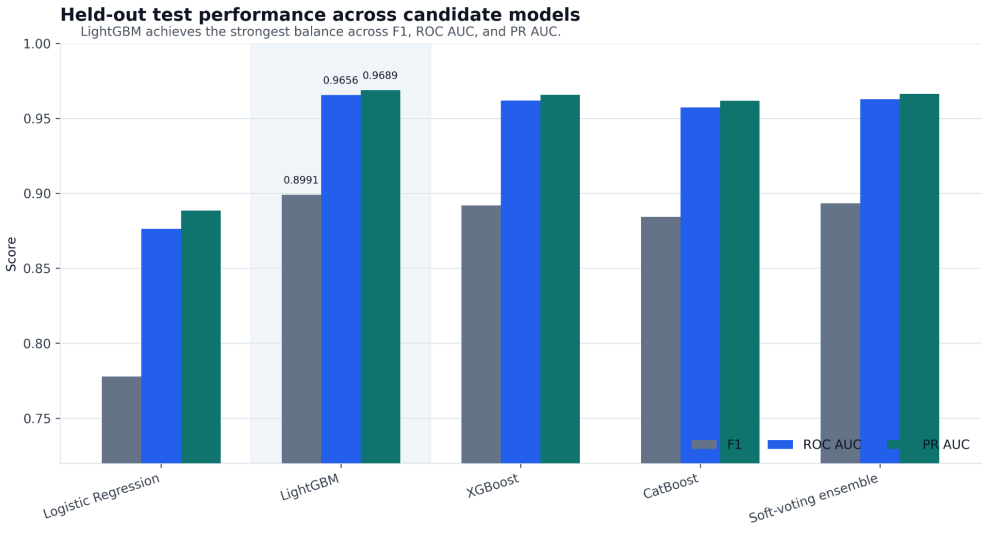


Figure 2. Held-out test performance comparison across candidate models.

The performance gap between Logistic Regression and the gradient boosting models is meaningful. Logistic Regression can only model a linear relationship between the URL features and the phishing label, while phishing indicators often appear through combinations of several features. For example, a long URL alone is not always malicious, and a high number of digits alone is not always enough to indicate phishing. However, a long URL with high entropy, multiple digits, complex subdomains, and unusual structural patterns becomes much more suspicious. This explains why tree-based models, especially LightGBM, perform better on this dataset.

The leakage ablation is the most important validation result. Without deduplication, a naive random split produces an inflated LightGBM F1-score of 0.9200 and ROC AUC of 0.9776. After deduplication, the honest test result becomes 0.8991 F1 and 0.9656 ROC AUC. As shown in Figure 3, duplicate leakage inflates the F1-score by +0.0208, or +2.3 percent relative to the honest result. This shows that the higher score from the naive split is not genuine improvement, but the result of identical feature vectors appearing in both training and testing partitions.

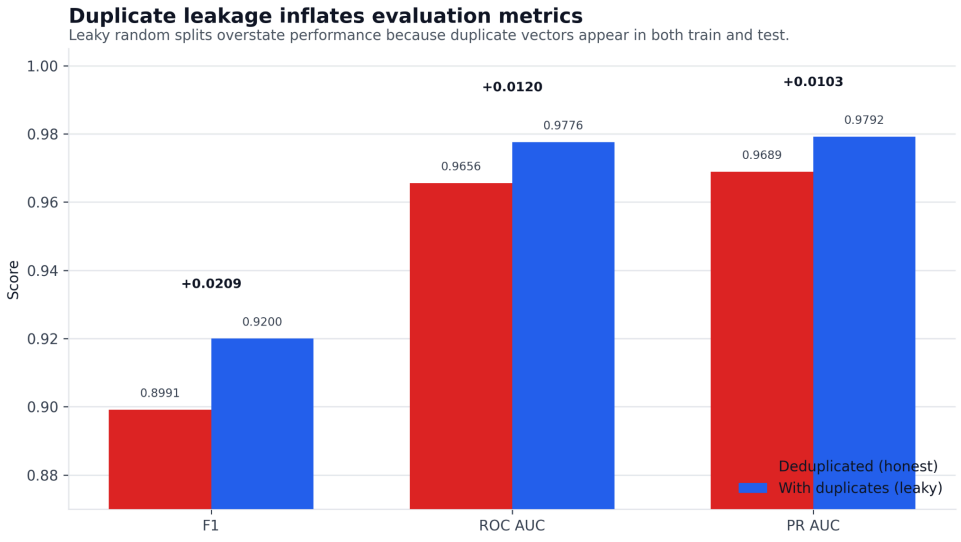


Figure 3. Effect of duplicate leakage on evaluation metrics.

This leakage finding is central to the study. A model intended for phishing detection should be evaluated based on its ability to detect unseen URLs, not repeated feature vectors that it has effectively already encountered during training. Therefore, the deduplicated evaluation is treated as the official and trustworthy result. Although the deduplicated score is lower than the leaky score, it better represents the model's expected behavior in real-world deployment.

The experiment also tested whether additional engineered ratio features improved performance. Four features were created: digit density, special character density, hyphen density, and subdomain depth ratio. These features were motivated by the assumption that the density of suspicious characters might carry more signal than their raw counts. However, the experiment shows that these engineered features do not improve the model. As shown in Table 2., the out-of-fold F1 slightly decreases from 0.9000 to 0.8998.

Feature Set	**Out-of-fold F1**
Raw 41 features	0.9000
Raw 41 features + 4 ratio features	0.8998
Difference	-0.0002

Table 2. Feature engineering ablation using LightGBM.

The result suggests that the original 41 URL-level features already contain enough signal for LightGBM. Since tree-based models can learn interactions between raw counts and URL length through split combinations, explicitly adding ratio features does not provide meaningful new information. Therefore, the final pipeline keeps the original 41-feature set to maintain a simpler and more stable model.

Finally, the decision threshold was tuned using out-of-fold predictions. The default threshold of 0.5 produced strong precision, but the selected threshold of 0.445 better matched the security objective. As shown in Table IV-2, the selected threshold improves recall from 0.8763 to

0.8917 and F1-score from 0.8991 to 0.9010. It also reduces false negatives from 1,559 to 1,365, meaning that around 194 additional phishing URLs are detected. Although false positives increase from 920 to 1,105, this trade-off is acceptable because false negatives are more dangerous in phishing detection.

Metric	Threshold 0.500	Threshold 0.445
Accuracy	0.9045	0.9048
Precision (phishing)	0.9231	0.9105
Recall (phishing)	0.8763	0.8917
F1 (phishing)	0.8991	0.9010
ROC AUC	0.9656	0.9656
PR AUC	0.9689	0.9689
False negatives	1,559	1,365
False positives	920	1,105

Table 3. Security-aware threshold comparison for the final LightGBM model.

The result suggests that the original 41 URL-level features already contain enough signal for LightGBM. Since tree-based models can learn interactions between raw counts and URL length through split combinations, explicitly adding ratio features does not provide meaningful new information. Therefore, the final pipeline keeps the original 41-feature set to maintain a simpler and more stable model.

Finally, the decision threshold was tuned using out-of-fold predictions. The default threshold of 0.5 produced strong precision, but the selected threshold of 0.445 better matched the security objective. As shown in Table IV-2, the selected threshold improves recall from 0.8763 to 0.8917 and F1-score from 0.8991 to 0.9010. It also reduces false negatives from 1,559 to 1,365, meaning that around 194 additional phishing URLs

are detected. Although false positives increase from 920 to 1,105, this trade-off is acceptable because false negatives are more dangerous in phishing detection.

4.2. Data Insights

The exploratory analysis shows that the dataset is suitable for supervised binary classification. The full dataset contains 128,541 legitimate URLs and 119,409 phishing URLs, or 51.84 percent and 48.16 percent respectively. After deduplication, the class distribution remains close to balanced, with 66,755 legitimate URLs and 63,023 phishing URLs. Therefore, no oversampling, undersampling, or class weighting is required, since the model can learn from both classes without severe imbalance.

The most important data insight is the high duplication rate. The dataset contains 118,172 exact duplicate rows, representing 47.66 percent of the original corpus. This occurs because the URLs are represented only through coarse lexical and structural features, so different URLs may collapse into identical feature vectors. This finding directly motivates the leak-free evaluation strategy used in this study: deduplication must be performed before splitting the data to avoid inflated performance.

The feature analysis also shows that phishing URLs tend to be more complex than legitimate URLs. They are generally longer, contain more digits and special characters, have higher entropy, and show more complex domain or subdomain structures. These patterns suggest that phishing URLs often use obfuscation, random-looking strings, or misleading structures to imitate legitimate services while hiding the actual destination.

In summary, the dataset supports the use of gradient-boosted decision trees. The classification signal is not driven by a single variable, but by interactions among URL length, entropy, digits, special characters, domain structure, and subdomain complexity. This explains why LightGBM substantially outperforms the linear Logistic Regression

baseline.

4.3. Model Interpretation

Model interpretation was conducted using SHAP values for the final LightGBM model. SHAP assigns each feature a contribution value that explains whether the feature pushes a prediction toward the phishing class or toward the legitimate class. This allows the model to be inspected beyond aggregate metrics.

The SHAP analysis shows that the most influential features are url_length, average_subdomain_length, domain_length, number_of_dots_in_url, number_of_subdomains, number_of_slash_in_url, number_of_digits_in_url, number_of_hyphens_in_url, entropy_of_domain, and entropy_of_url. The top feature is url_length, with the highest mean absolute SHAP value, indicating that the overall length of the URL is the strongest driver of the final LightGBM predictions. The ranking is shown in Figure 4.

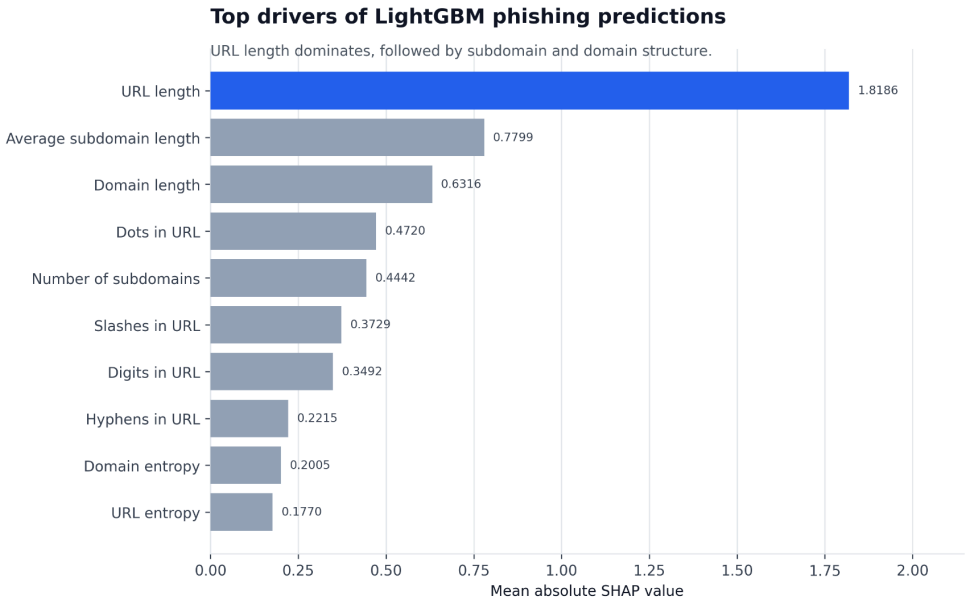


Figure 4. Top drivers of LightGBM phishing predictions based on mean absolute SHAP value.

Chapter V Conclusion and Recommendations

5.1. Conclusion

This study developed a leak-free tabular classification pipeline for phishing URL detection using only intra-URL lexical and structural features. The dataset consists of 247,950 labeled URLs represented by 41 features and one binary target. Since the model does not require webpage content, HTML, screenshots, or external server access, the proposed approach remains lightweight, privacy-preserving, and suitable for resource-constrained environments.

The main methodological finding is the presence of substantial duplication in the dataset. A total of 118,172 rows, or 47.66 percent of the corpus, are exact duplicates. If these duplicates are not removed before splitting, the model evaluation becomes inflated because identical feature vectors may appear in both training and testing sets. The leakage ablation confirms this issue: a naive split increases the LightGBM F1-score from 0.8991 to 0.9200. Therefore, deduplication before splitting is essential for obtaining an honest generalization estimate.

Among all evaluated models, LightGBM achieved the strongest performance. At the default threshold of 0.5, it reached an F1-score of 0.8991, ROC AUC of 0.9656, accuracy of 0.9045, precision of 0.9231, recall of 0.8763, and PR AUC of 0.9689. After threshold tuning, the selected operating threshold of 0.445 improved phishing recall to 0.8917 and F1-score to 0.9010, reducing missed phishing URLs from 1,559 to 1,365.

Overall, this research shows that a carefully validated gradient-boosted decision tree model can provide strong and interpretable phishing detection using lightweight URL-level features. More importantly, it demonstrates that honest evaluation is as important as model performance, especially in a security context where inflated metrics can lead to misleading conclusions.

5.2. Recommendations

Future work should validate the model on newer phishing datasets to test robustness against evolving attack patterns. The feature set can also be expanded with safe external signals, such as domain age, DNS metadata, SSL certificate information, or reputation indicators, while still avoiding full webpage rendering.

In addition, the model can be developed into a real-time prototype, such as a browser extension, email filter, local gateway, or API-based detection service. Future development should also explore adaptive thresholding so the system can adjust the trade-off between false positives and false negatives based on deployment needs.

References

[1] World Economic Forum, "Why the humanitarian sector's cybersecurity is a priority," Jan. 2022.

[2] World Economic Forum, "Target-rich, cyber-poor: How to strengthen cybersecurity and build resilience in vulnerable sectors," Global Cybersecurity Outlook 2026, 2026.

[3] CyberPeace Institute, "CyberPeace Institute launches Humanitarian Cybersecurity Center (HCC)," Feb. 2023.

[4] Faculty of Information Technology, "FIT Competition 2026 Guidebook: Digital Impact for Humanitarian Response and Global Well-Being," Satya Wacana Christian University, 2026.

[5] K. Hayat and B. Magnier, "Data Leakage and Deceptive Performance: A Critical Examination of Credit Card Fraud Detection Methodologies," Mathematics, vol. 13, no. 16, p. 2563, 2025.

[6] T. D. Hemanth Gowda et al., "PhreshPhish: A Comprehensive Phishing Dataset," arXiv:2507.10854, Jul. 2025.

[7] M. Vero, M. Balunovic, D. I. Dimitrov, and M. Vechev, "TabLeak: Tabular Data Leakage in Federated Learning," in Proc. 40th Int. Conf. Mach. Learn., 2023, pp. 35051-35083.

[8] D. Sahoo, C. Liu, and S. C. H. Hoi, "Malicious URL Detection using Machine Learning: A Survey," arXiv:1701.07179, Jan. 2017.

[9] A. Martin et al., "Digitisation and Sovereignty in Humanitarian Space: Technologies, Territories and Tensions," Geopolitics, vol. 28, no. 3, pp. 1362-1397, 2023.

Appendix

A. Source Code

The complete and reproducible analysis is contained in the accompanying Jupyter notebook, Notebook_Preliminary_IndomaretLabtekV.ipynb. The notebook fixes a single global random seed, locates the data file automatically, and runs from the first cell to the last without manual intervention, reproducing every number and figure reported here.

B. Additional Figures

Figure 7 shows the receiver operating characteristic curve and the precision-recall curve for the final LightGBM model on the held out test set, summarized by the ROC AUC of 0.9656 and the average precision of 0.969.

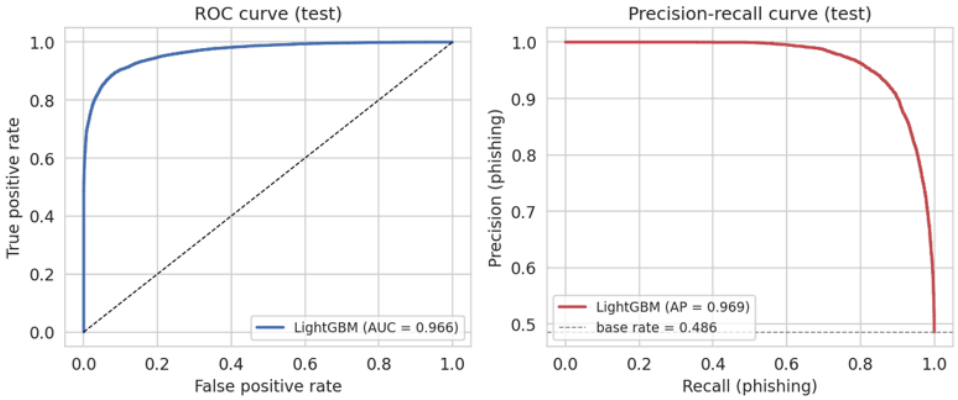


Figure 5. ROC curve and precision-recall curve for the final LightGBM model on the held-out test set.

C. Full Feature List by Category

URL level lexical counts (15): url_length, number_of_dots_in_url, having_repeated_digits_in_url, number_of_digits_in_url, number_of_special_char_in_url, number_of_hyphens_in_url, number_of_underline_in_url, number_of_slash_in_url, number_of_questionmark_in_url, number_of_equal_in_url, number_of_at_in_url, number_of_dollar_in_url,

number_of_exclamation_in_url, number_of_hashtag_in_url, number_of_percent_in_url.

Domain level (8): domain_length, number_of_dots_in_domain, number_of_hyphens_in_domain, having_special_characters_in_domain, number_of_special_characters_in_domain, having_digits_in_domain, number_of_digits_in_domain, having_repeated_digits_in_domain.

Subdomain level (11): number_of_subdomains, having_dot_in_subdomain, having_hyphen_in_subdomain, average_subdomain_length, average_number_of_dots_in_subdomain, average_number_of_hyphens_in_subdomain, having_special_characters_in_subdomain, number_of_special_characters_in_subdomain, having_digits_in_subdomain, number_of_digits_in_subdomain, having_repeated_digits_in_subdomain.

Path, query, fragment, and anchor (5): having_path, path_length, having_query, having_fragment, having_anchor.

Entropy (2): entropy_of_url, entropy_of_domain.

Target (1): Type (0 for legitimate, 1 for phishing).